SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PYTHON PROGRAMMING – ASSIGNMENT 2

Course Code: CSA0823

Assignment Title: Analysis, Implementation and Evaluation of Python Programming Concepts

Submitted by:

Name: D.S.Tharuniya Priya

Register Number: 192511317

Programme: B.E. Computer Science and Engineering

Year: II Year

Submitted to:

Faculty Name: Hariramachandran

Department of Computer Science and Engineering

SIMATS Engineering

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Question 1 – Decisions with Many Boundaries
2. Question 2 – Loops That Count Attempts
3. Question 3 – Vehicle Registration Number Validator
4. Question 4 – Building Long Text: + or join()
5. Question 5 – Functions, Parameters and Scope
6. Question 6 – Recursion and Its Limits
7. Question 7 – Diagnose, Redesign and Validate

QUESTION 1 – DECISIONS WITH MANY BOUNDARIES

Marks: 6

1.1 Aim

To implement and analyse electricity subsidy eligibility using separate if statements, if-elif-else and match-case statements. The objective is to understand how conditional statements handle multiple boundaries and prevent overlapping decisions.

1.2 Problem Statement

A household is eligible for an electricity subsidy according to its electricity consumption and eligibility conditions.

The rules are:

1. If electricity consumption is 100 units or less, the household receives a full subsidy.
2. If consumption is between 101 and 200 units, the household receives a partial subsidy only if it holds a ration card.
3. If consumption is above 200 units, no subsidy is provided unless the household has a member above 80 years of age.
4. All other households receive no subsidy.

1.3 Prediction Before Execution

Before executing the program, the following results are predicted for eight households.

Household	Units	Ration Card	Member Above 80	Predicted Decision
H1	100	Yes	No	Full subsidy
H2	101	Yes	No	Partial subsidy
H3	101	No	No	No subsidy
H4	150	Yes	No	Partial subsidy
H5	200	Yes	No	Partial subsidy
H6	200	No	No	No subsidy

H7	201	No	Yes	Special subsidy
H8	250	Yes	No	No subsidy

The boundary values 100, 101, 200 and 201 are included to verify that the conditions work correctly.

1.4 Implementation Using Separate if Statements

In separate if statements, every condition is evaluated independently. Therefore, if conditions overlap, more than one decision may be produced.

For demonstration, the incorrect implementation uses a partial-subsidy condition of consumption less than or equal to 200 along with ration-card eligibility. This causes households consuming 100 units to satisfy both the full and partial subsidy conditions.

1.5 Implementation Using if-elif-else

The if-elif-else structure evaluates conditions sequentially. Once one condition becomes true, the remaining conditions are skipped.

This ensures that only one decision is returned for each household.

The chained comparison 101 <= units <= 200 clearly represents the required consumption range.

1.6 Observed Results

Household	**Units**	**Separate if Result**	**if-elif-else Result**
H1	100	Full + Partial	Full
H2	101	Partial	Partial
H3	101	No subsidy	No subsidy
H4	150	Partial	Partial
H5	200	Partial	Partial
H6	200	No subsidy	No subsidy
H7	201	Special	Special

H8	250	No subsidy	No subsidy

The separate-if implementation produces two results for H1 because its conditions overlap. The if-elif-else implementation produces exactly one decision.

1.7 Design Decision: Nested if or Combined Boolean Conditions

Combined Boolean conditions using and and chained comparisons are selected for this problem.

For example:

101 <= units <= 200 and ration_card

This approach is easier to understand because the eligibility range and the ration-card requirement are expressed in the same condition.

Nested if statements would introduce additional indentation and make the program longer. Combined conditions are more suitable when the rules are simple and directly related.

1.8 Alternative Concept: match-case

Python 3.10 introduced the match-case statement.

The match-case statement can be used with guarded patterns to check electricity consumption and eligibility conditions.

It provides an alternative way of expressing multiple decisions. However, for this particular problem, if-elif-else is more familiar and straightforward.

1.9 Time and Space Complexity Analysis

Let k represent the number of electricity consumption slabs.

Approach	**Time Complexity**	**Auxiliary Space**
Separate if statements	O(k)	O(1)
if-elif-else	O(k) worst case	O(1)
match-case	O(k) worst case	O(1)
Lookup using a list and loop	O(k) worst case	O(k) for the stored lookup table

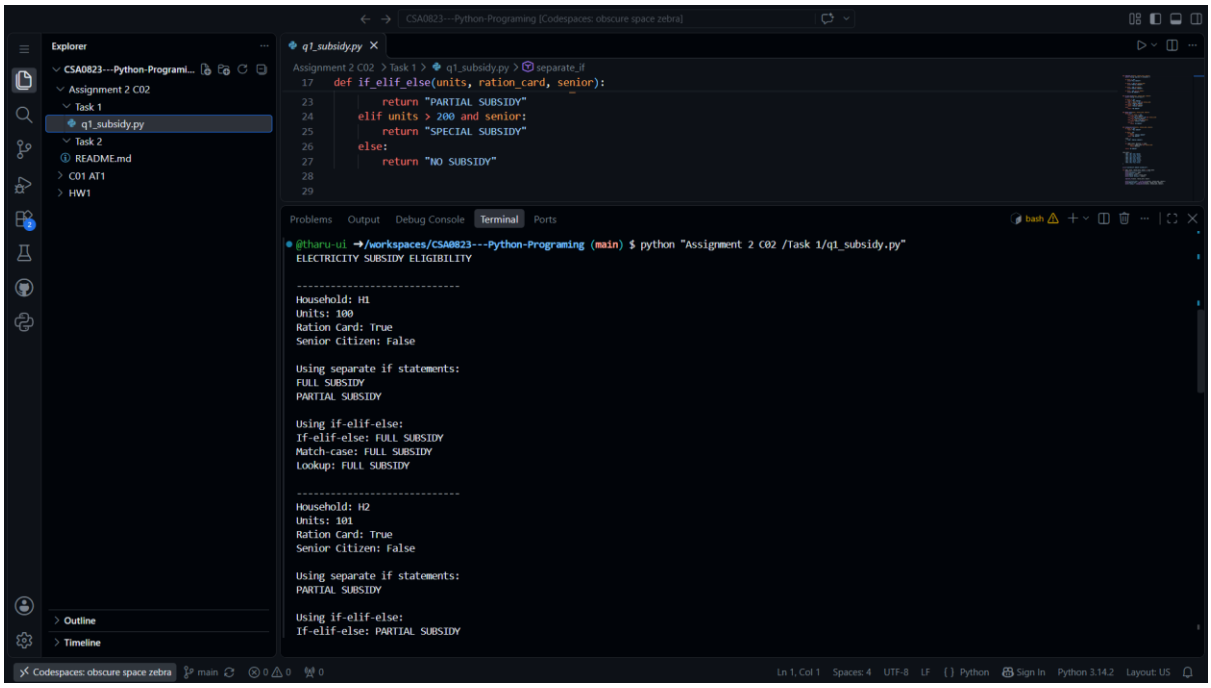
For the given problem, k is fixed. Therefore, the execution time is effectively constant for each household.

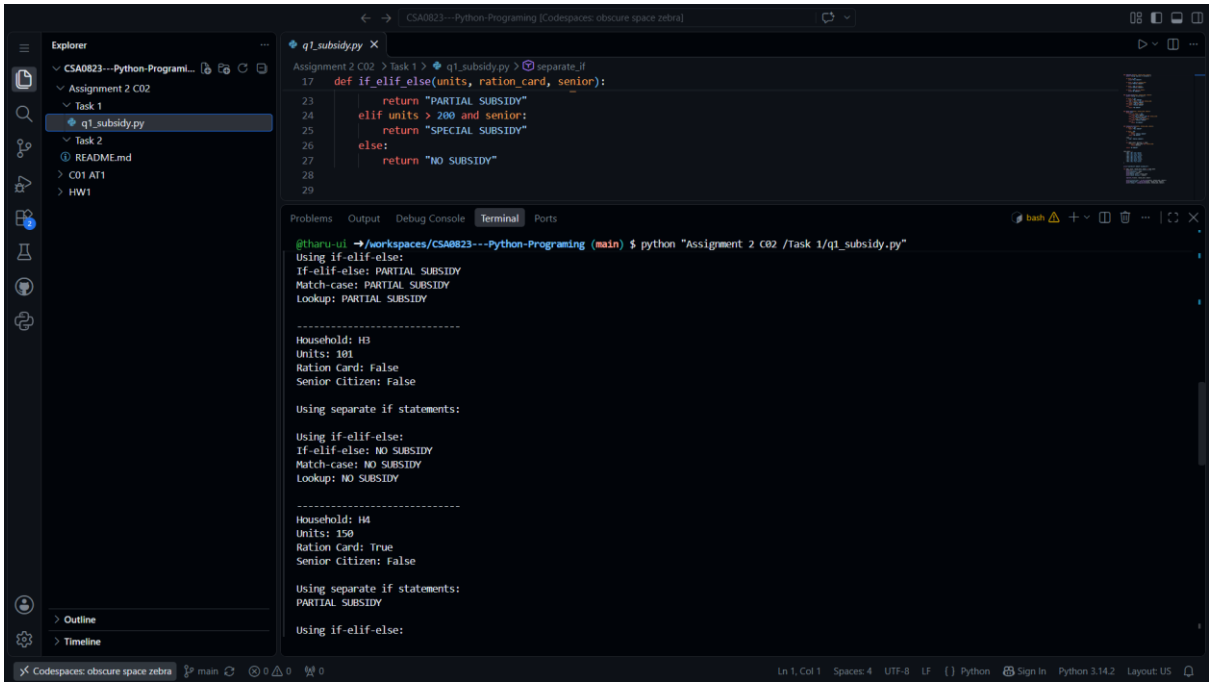
The lookup approach requires additional storage for the slab information. The conditional approaches do not require a separate lookup table.

1.10 Conclusion

The experiment demonstrates that separate if statements can produce multiple decisions when their conditions overlap. The if-elif-else structure is more appropriate for mutually exclusive eligibility categories because it ensures that only one decision is selected. Combined Boolean conditions also improve readability and simplify future modifications.

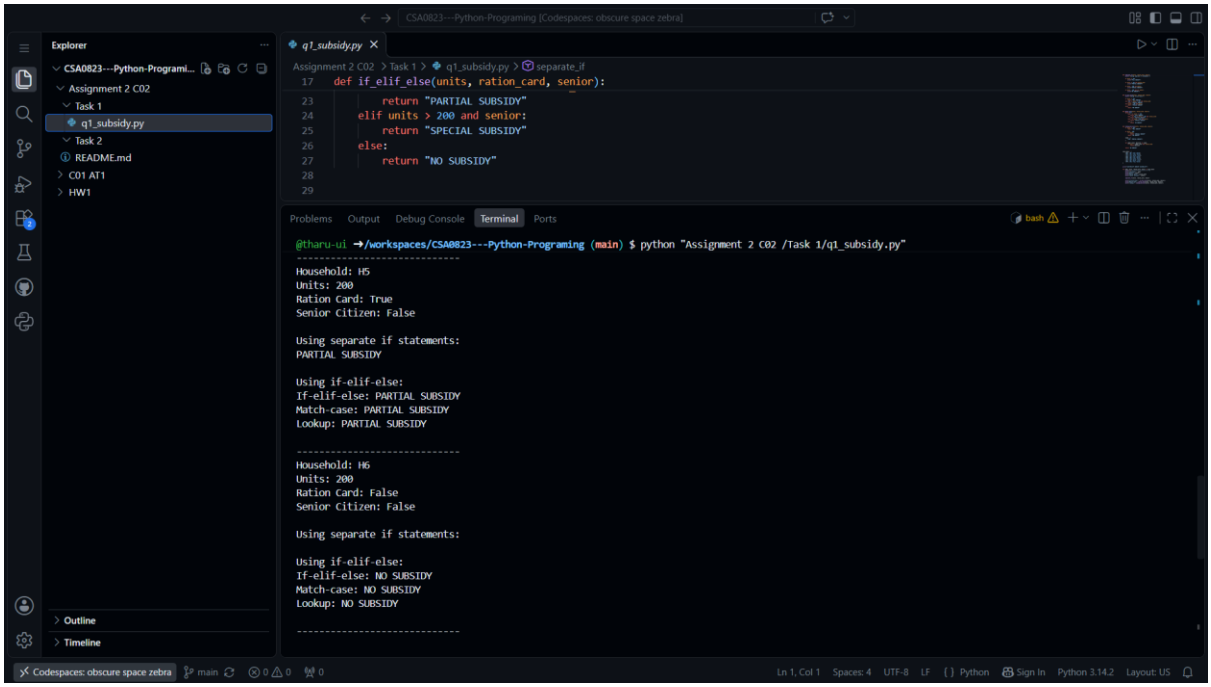
Output Screenshot :




CSA0823---Python-Programing [Codespaces: obscure space zebra]
Explorer
CSA0823---Python-Programi...
Assignment 2 C02
Task 1
q1_subsidy.py
Task 2
README.md
C01 AT1
HW1
Outline
Timeline
q1_subsidy.py
Assignment 2 C02 > Task 1 > q1_subsidy.py > separate_if
17 def if_elif_else(units, ration_card, senior):
23 return "PARTIAL SUBSIDY"
24 elif units > 200 and senior:
25 return "SPECIAL SUBSIDY"
26 else:
27 return "NO SUBSIDY"
28
29
Problems Output Debug Console Terminal Ports
bash
@tharu-ui ➜ /workspaces/CSA0823---Python-Programing (main) $ python "Assignment 2 C02 /Task 1/q1_subsidy.py"
Using if-elif-else:
If-elif-else: PARTIAL SUBSIDY
Match-case: PARTIAL SUBSIDY
Lookup: PARTIAL SUBSIDY

Household: H3
Units: 101
Ration Card: False
Senior Citizen: False
Using separate if statements:
Using if-elif-else:
If-elif-else: NO SUBSIDY
Match-case: NO SUBSIDY
Lookup: NO SUBSIDY

Household: H4
Units: 150
Ration Card: True
Senior Citizen: False
Using separate if statements:
PARTIAL SUBSIDY
Using if-elif-else:
Codespaces: obscure space zebra
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US


CSA0823---Python-Programing [Codespaces: obscure space zebra]
Explorer
CSA0823---Python-Programi...
Assignment 2 C02
Task 1
q1_subsidy.py
Task 2
README.md
C01 AT1
HW1
Outline
Timeline
q1_subsidy.py
Assignment 2 C02 > Task 1 > q1_subsidy.py > separate_if
17 def if_elif_else(units, ration_card, senior):
23 return "PARTIAL SUBSIDY"
24 elif units > 200 and senior:
25 return "SPECIAL SUBSIDY"
26 else:
27 return "NO SUBSIDY"
28
29
Problems Output Debug Console Terminal Ports
bash
@tharu-ui ➜ /workspaces/CSA0823---Python-Programing (main) $ python "Assignment 2 C02 /Task 1/q1_subsidy.py"

Household: H5
Units: 200
Ration Card: True
Senior Citizen: False
Using separate if statements:
PARTIAL SUBSIDY
Using if-elif-else:
If-elif-else: PARTIAL SUBSIDY
Match-case: PARTIAL SUBSIDY
Lookup: PARTIAL SUBSIDY

Household: H6
Units: 200
Ration Card: False
Senior Citizen: False
Using separate if statements:
Using if-elif-else:
If-elif-else: NO SUBSIDY
Match-case: NO SUBSIDY
Lookup: NO SUBSIDY

Codespaces: obscure space zebra
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US

QUESTION 2 – LOOPS THAT COUNT ATTEMPTS

Marks: 4

2.1 Aim

To implement an OTP verification system using while and for loops and understand the use of break, continue, pass and loop-else.

2.2 Problem Statement

A bank kiosk allows a customer three attempts to enter the correct OTP. If the customer enters the wrong OTP three times, the account is locked for 30 minutes.

A blank entry must not count as an attempt.

2.3 Algorithm

1. Initialise the correct OTP and attempt counter.
2. Accept the OTP entered by the customer.
3. If the input is blank, display a message and request the OTP again without increasing the attempt count.
4. If the OTP is correct, display successful verification and terminate the loop using break.
5. If the OTP is incorrect, increase the attempt count.
6. If all three attempts are unsuccessful, execute the loop-else block and lock the account.

2.4 Implementation Using while Loop

The while loop is suitable because the number of attempts depends on a condition. The loop continues until the correct OTP is entered or the maximum number of attempts is reached.

The attempt counter is increased only after a non-blank entry.

2.5 Implementation Using for Loop

The for loop uses range(3) to provide exactly three valid OTP attempts.

An inner while loop handles blank inputs. This ensures that blank entries do not consume one of the three attempts.

2.6 Use of break, continue, pass and Loop-else

Statement	Purpose	Application
break	Terminates the loop	Stops verification when the correct OTP is entered
continue	Skips the remaining statements in the current iteration	Rejects blank input without counting an attempt
pass	Performs no operation	Acts as a placeholder where no additional action is required
loop-else	Executes when the loop finishes without break	Locks the account after all attempts fail

The pass statement is demonstrated as a placeholder in the successful verification branch. It does not change the program's execution.

2.7 Changed Condition: Two Extra Attempts

The bank introduces a security question. If the customer answers the security question correctly, two additional OTP attempts are provided.

Therefore:

- Normal maximum attempts = 3.
- Maximum attempts after successful security verification = 5.

The loop limit can be changed from 3 to 5 after successful security verification.

The same loop structure can be reused without duplicating the OTP verification logic.

2.8 Comparison of while and for Loops

Feature	while Loop	for Loop
Control	Condition-based	Sequence-based
Attempt count	Manually maintained	Controlled by range

Suitable when	Number of attempts changes dynamically	Number of attempts is fixed
Counter management	Requires manual updating	Automatically advances
Common error	Incorrect counter update	Incorrect range value

A while loop is more natural when the number of attempts is determined dynamically.

A common bug occurs when the attempt counter is increased before checking for blank input. This causes blank entries to consume attempts. Another bug occurs when the counter is never updated, which may create an infinite loop.

2.9 Alternative Concept: Recursion

The OTP verification process can also be implemented using a recursive function.

The function accepts the remaining number of attempts as a parameter. If the OTP is correct, it returns success. If the OTP is incorrect, it calls itself with one fewer attempt.

When the remaining attempts become zero, the function locks the account.

Recursion is possible but less suitable because it consumes additional call-stack memory.

2.10 Complexity Analysis

Let a represent the maximum number of valid OTP attempts.

Approach	**Time Complexity**	**Auxiliary Space**
while loop	O(a)	O(1)
for loop	O(a)	O(1)
Recursive approach	O(a)	O(a)

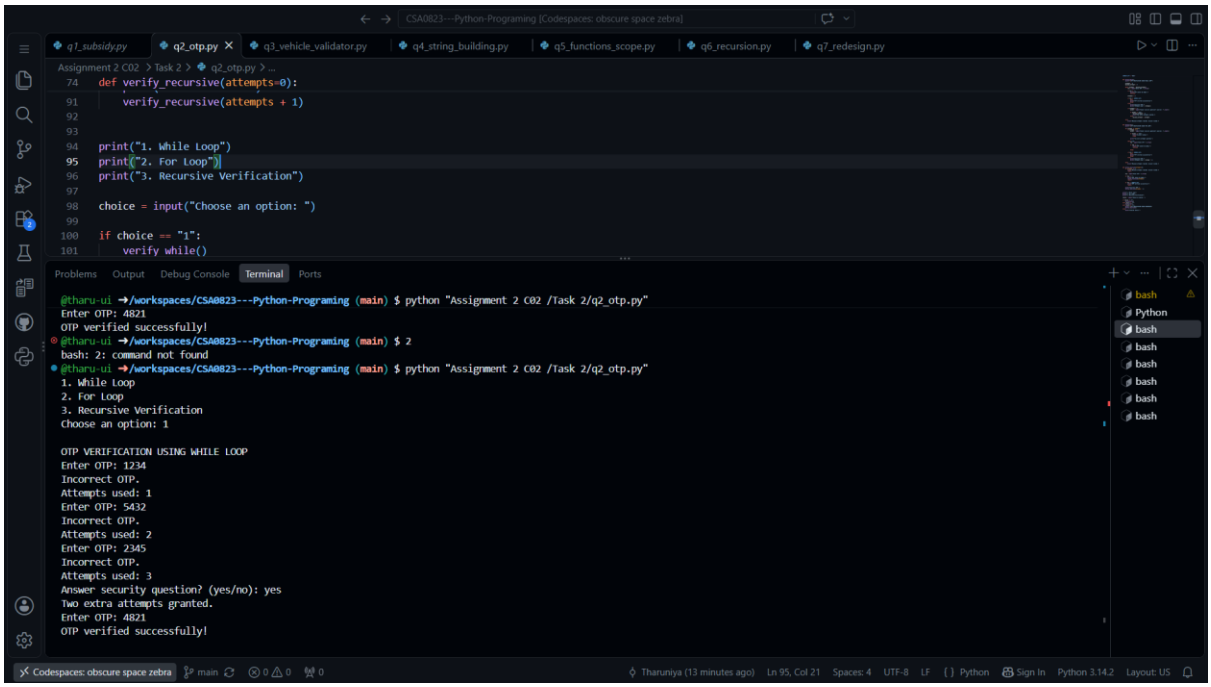
The loop-based solutions require constant auxiliary memory, whereas recursion requires additional stack space for each function call.

2.11 Conclusion

Both while and for loops can implement OTP verification correctly. The while loop is more flexible for condition-based repetition, while the for loop is convenient when the number of

attempts is fixed. Blank inputs must be handled without increasing the attempt count. Loop-else provides a clear way to handle account locking after unsuccessful attempts.

Output Screenshot :



QUESTION 3 – VEHICLE REGISTRATION NUMBER VALIDATOR

Marks: 6

3.1 Aim

To develop a Python program that normalises, validates and formats vehicle registration numbers using string operations and slicing without using regular expressions.

3.2 Problem Statement

A toll plaza receives vehicle registration numbers entered in different formats.

Examples:

- tn 09 ab 1234
- TN-09-AB-1234
- Tn09Ab1234

The program must convert these different representations into one standard format and check whether the registration number follows the required structure.

3.3 Registration Number Structure

A valid registration number consists of:

Component	Length	Description
State code	2	Alphabetic characters
District code	2	Numeric characters
Series	1 or 2	Alphabetic characters
Vehicle number	4	Numeric characters

Therefore, the total length must be either 9 or 10 characters.

3.4 Prediction Before Execution

Input	Predicted Normalised Form	Prediction

tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
Tn09Ab1234	TN09AB1234	Valid
TN9AB1234	TN9AB1234	Invalid
TN 22 1234 AB	TN221234AB	Invalid
TN-09-ABC-1234	TN09ABC1234	Invalid

3.5 String Normalisation

The normalise() function performs the following operations:

1. strip() removes leading and trailing spaces.
2. replace() removes spaces.
3. replace() removes hyphens.
4. upper() converts lowercase letters into uppercase.

The function returns the normalised registration number.

3.6 Validation Using Slicing

The is_valid() function first checks whether the length is 9 or 10.

For a 9-character registration number:

- reg[0:2] represents the state code.
- reg[2:4] represents the district code.
- reg[4:5] represents the one-letter series.
- reg[5:9] represents the four-digit vehicle number.

For a 10-character registration number:

- reg[0:2] represents the state code.
- reg[2:4] represents the district code.
- reg[4:6] represents the two-letter series.

- reg[6:10] represents the four-digit vehicle number.

The isalpha() and isdigit() methods verify the character types.

3.7 Receipt Formatting

A valid registration number is formatted using slicing.

The first two characters represent the state code, the next two represent the district code, the middle characters represent the series, and the last four represent the vehicle number.

The output format is:

TN 09 AB 1234

3.8 Inquiry: String Immutability

Strings in Python are immutable. This means that their contents cannot be modified after the string object has been created.

When an operation appears to change a string, Python creates a new string instead of modifying the original string.

For example:

text = "hello"

text.upper() returns "HELLO", but the original variable still refers to "hello" unless the result is assigned back.

Therefore:

text = text.upper()

is required to store the uppercase result.

3.9 Explanation of Slicing

For a 9-character string:

- s[-4:] returns the last four characters.
- s[4:-4] returns the characters between index 4 and the last four characters.

For a 10-character string, the same expressions work because the middle series contains two characters instead of one.

Example:

For TN09AB1234:

- s[-4:] returns 1234.
- s[4:-4] returns AB.

For TN09A1234:

- s[-4:] returns 1234.
- s[4:-4] returns A.

3.10 Alternative Concept: Regular Expressions

The same validation can be performed using Python's re module.

The expression:

[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}

represents two letters, two digits, one or two letters and four digits.

Using re.fullmatch() ensures that the complete registration number matches the specified structure.

Regular expressions are shorter and convenient for complex patterns, whereas slicing and string methods are easier for beginners to understand step by step.

3.11 Complexity Analysis

Let L represent the length of the registration number.

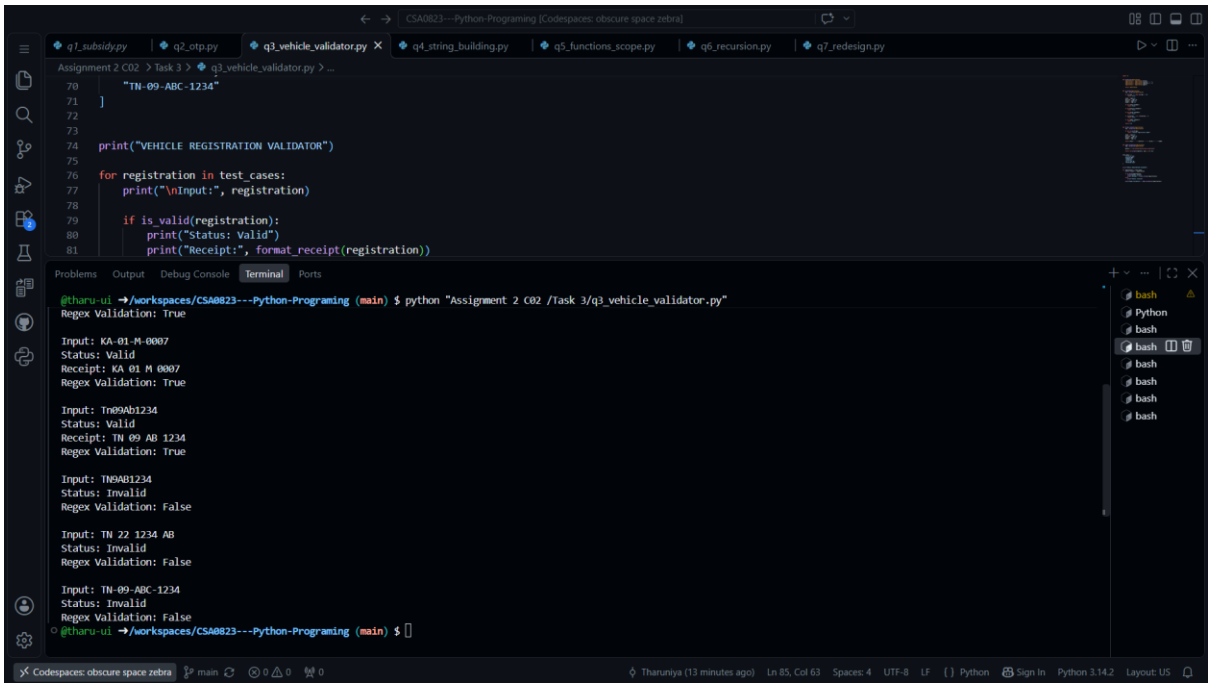
Approach	Time Complexity	Auxiliary Space
String operations and slicing	O(L)	O(L)
Regular expression validation	O(L) for this fixed pattern	O(L) worst case, including internal matching/storage

Both approaches process a string whose length is proportional to L.

3.12 Conclusion

The program successfully normalises different registration number formats, validates the required structure and produces a standard receipt format. String slicing and string methods provide a simple solution without requiring regular expressions.

Output Screenshot :



QUESTION 4 – BUILDING LONG TEXT: + OR join()?

Marks: 6

4.1 Aim

To compare repeated string concatenation, list-based join() and f-string-based statement construction using execution-time measurements.

4.2 Problem Statement

A mobile operator generates monthly statements containing a large number of call and SMS records.

When repeated string concatenation is used inside a loop, statement generation can become slow for large inputs.

The objective is to identify an efficient method for constructing long strings.

4.3 Prediction Before Execution

The predicted performance order is:

1. join() is expected to be efficient because it combines all string elements in a single operation.
2. F-string formatting followed by join() is expected to perform well, although formatting introduces additional work.
3. Repeated += may be slower for large inputs because strings are immutable and repeated concatenation may require creating new string objects.

CPython may optimise some concatenation patterns, so actual execution times must be measured.

4.4 Implementation Methods

Method A – Repeated +=

A string is initialised as an empty string. Each item is appended using the += operator inside a loop.

Method B – List and join()

All statement components are stored in a list. The join() method combines the elements into a single string.

Method C – F-string

F-strings are used to construct individual statement lines. The resulting strings are collected and combined using join().

4.5 Experimental Procedure

The programs are tested using three input sizes:

- 1,000 lines
- 10,000 lines
- 200,000 lines

Each method is executed three times for every input size.

The time.perf_counter() function measures the execution duration.

The average time is calculated using:

Average time = (Run 1 + Run 2 + Run 3) / 3

4.6 Experimental Results

Number of Lines	Repeated += Average (s)	join() Average (s)	F-string + join() Average (s)
1,000	0.000444	0.000299	0.000276
10,000	0.004144	0.003080	0.002884
200,000	0.186266	0.120280	0.071510

The output equality check confirms whether the methods generate identical statements.

4.7 Explanation Using String Immutability

Python strings cannot be modified after creation.

When repeated concatenation is performed, Python may need to create a new string containing the previous contents and the newly appended text.

As the statement becomes longer, repeated copying can increase the amount of work.

The join() method is generally recommended because it can calculate the required output size and combine the elements efficiently.

CPython sometimes optimises += operations, so the measured difference may vary depending on the Python implementation, input size and execution environment.

4.8 Alternative Concept: File Writing and io.StringIO

Instead of constructing the entire statement in memory, the program can write each line directly to a file.

Another option is io.StringIO, which provides an in-memory file-like object for efficient text construction.

Direct file writing is useful when the output is very large and does not need to remain entirely in memory.

StringIO is useful when an in-memory result is required but repeated concatenation should be avoided.

4.9 Complexity Analysis

Let n represent the number of statement lines and assume the average line length is bounded.

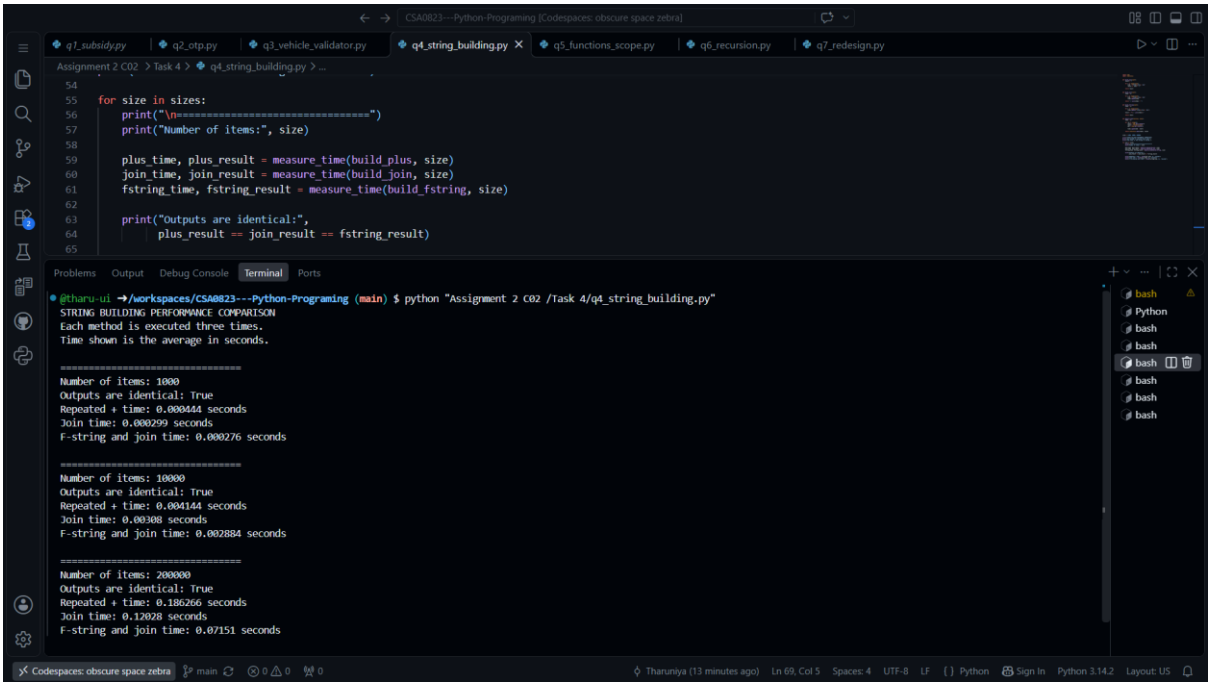
Method	**Time Complexity**	**Memory Requirement**
Repeated +=	Up to $O(n^2)$	Up to O(n) output storage
List and join()	O(n)	O(n) for the list and final string
F-string and join()	O(n)	O(n) for formatted parts and final string
Direct file writing	O(n)	O(1) auxiliary memory, excluding buffering
StringIO	O(n)	O(n) for the accumulated text

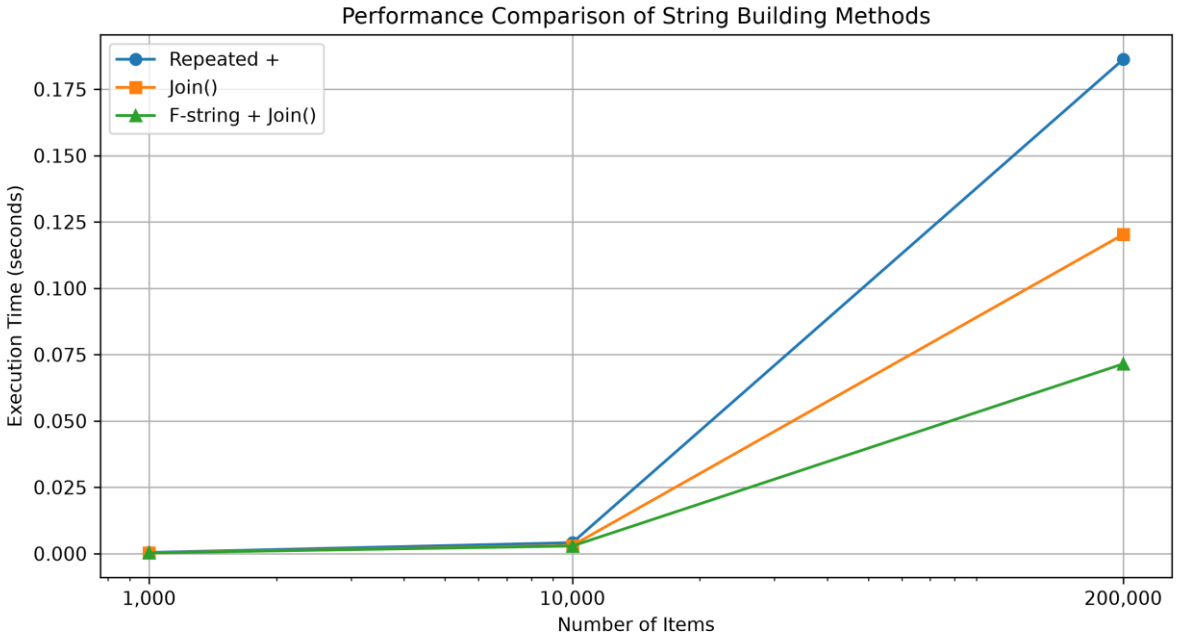
The actual amount of memory depends on the total number of characters, not just the number of lines.

4.10 Conclusion

The experiment compares three methods of building long strings. The join() method is generally preferred when many string elements must be combined. Direct file writing is more suitable when a very large statement does not need to be retained entirely in memory.

Output Screenshot :





QUESTION 5 – FUNCTIONS, PARAMETERS AND SCOPE

Marks: 7

5.1 Aim

To understand Python function parameters, mutable default arguments, returning multiple values and variable scope using a college canteen order system.

5.2 Problem Statement

A canteen order program uses a function with a mutable default list.

The function is:

def add_item(item, order=[]):

The default list is created only once when the function is defined. Therefore, subsequent calls without an explicit order argument reuse the same list.

This causes items from previous customers to appear in later customers' orders.

5.3 Explanation of the Mutable Default Argument Bug

Default parameter values are evaluated once when a function is defined, not every time it is called.

In the buggy function, the same list is shared between calls.

For example:

- First call adds Tea.
- Second call adds Vada to the same list.
- The resulting list contains both items.

This is incorrect when separate customers should have independent orders.

5.4 Correct Function Design

The corrected function uses None as the default value.

When the function receives None, it creates a new empty list.

Each customer therefore receives an independent order list unless an existing list is explicitly passed.

5.5 Types of Function Parameters

Positional Parameters

Arguments are passed according to their position in the function definition.

Example:

add_item("Tea", order)

Keyword Parameters

Arguments are passed using their parameter names.

Example:

calculate_bill(amount=1000, gst_rate=0.05)

Keyword parameters improve readability.

Default Parameters

A default value is provided in the function definition.

Example:

def calculate_bill(amount, gst_rate=0.05):

If the GST rate is not provided, the default rate is used.

5.6 Returning Multiple Values

Python functions can return more than one value by separating the values with commas.

Example:

return total, gst

The returned values can be unpacked into separate variables.

total, gst = calculate_bill(1000)

This allows the program to receive both the total amount and the GST amount from one function.

5.7 Local, Global and Nonlocal Scope

Local Scope

A local variable is declared inside a function and can only be accessed within that function.

Global Scope

A global variable is declared outside functions and can be accessed throughout the module.

To reassign a global variable inside a function, the global keyword is required.

Nonlocal Scope

The nonlocal keyword is used inside a nested function to refer to a variable in the nearest enclosing function scope.

It does not refer to a global variable.

5.8 Why Global Variables Are Usually Avoided

Global variables can be modified by multiple functions, making it difficult to identify where a value has changed.

This can create unexpected behaviour and make debugging difficult.

Passing values through function parameters and returning results is generally a better design because it makes the program easier to test and maintain.

5.9 Test Results for Three Customers

Customer	Items Ordered	Expected Result
Customer 1	Tea, Vada	Independent order
Customer 2	Coffee	Independent order
Customer 3	Juice	Independent order

The corrected function creates a separate list for each customer.

The buggy function reuses the same default list.

5.10 Alternative Concept: Explicit Arguments and Closures

An alternative design is to require the caller to pass an order list explicitly.

Another alternative is to use a closure.

A closure can maintain an order list inside an outer function and provide an inner function to add items.

This keeps the order data private to the customer-specific function.

5.11 Complexity Analysis

Let n represent the number of items in a customer's order.

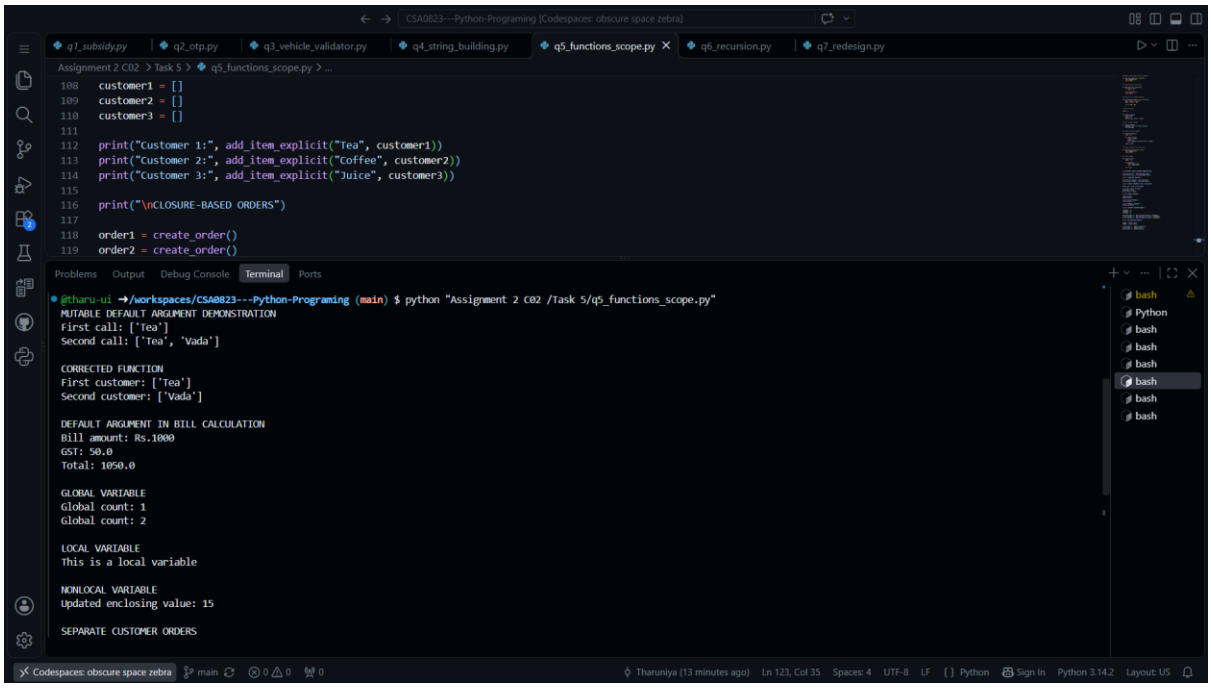
Approach	Time per Item	Space
Mutable default list	O(1) amortised	O(n) shared list
None default with new list	O(1) amortised	O(n) per customer
Explicit list argument	O(1) amortised	O(n) per customer
Closure-based list	O(1) amortised	O(n) per closure

The corrected design provides independent order storage for each customer.

5.12 Conclusion

The experiment demonstrates that mutable default arguments can cause data to be shared unintentionally between function calls. Using None as the default value and creating a new list when required prevents this problem. Understanding local, global and nonlocal scope also helps in developing maintainable Python programs.

Output Screenshot :



QUESTION 6 – RECURSION AND ITS LIMITS

Marks: 8

6.1 Aim

To implement staircase counting using recursion, analyse repeated function calls, apply memoisation and compare recursive solutions with an iterative approach.

6.2 Problem Statement

A devotee climbs a temple staircase containing n steps.

At each movement, the devotee can climb either one step or two steps.

The objective is to calculate the total number of distinct ways to reach the top.

6.3 Recursive Relationship

The number of ways to reach step n is the sum of:

- The number of ways to reach step n-1.
- The number of ways to reach step n-2.

Therefore:

ways(n) = ways(n-1) + ways(n-2)

Base Cases

ways(0) = 1

ways(1) = 1

The first base case represents one way to remain at the starting position. The second represents one way to climb a single step.

6.4 Hand Calculation

Number of Steps	Number of Ways
1	1
2	2

3	3
4	5
5	8
6	13

The results follow the Fibonacci sequence with the initial values 1 and 1.

6.5 Call Tree for n = 5

The naive recursive function generates the following call tree:

ways(5)

- ways(4)
    - ways(3)
        - ways(2)
            - ways(1)
            - ways(0)
        - ways(1)
    - ways(2)
        - ways(1)
        - ways(0)
- ways(3)
    - ways(2)
        - ways(1)
        - ways(0)
    - ways(1)

The same subproblems are calculated repeatedly.

For example, ways(3), ways(2) and ways(1) occur multiple times.

This repeated calculation increases execution time.

6.6 Naive Recursion

The naive recursive solution directly implements the mathematical relationship.

Although it is simple, it performs repeated calculations because previously computed results are not stored.

Its time complexity is exponential.

6.7 Memoisation

Memoisation stores previously calculated results in a cache.

The functools.lru_cache decorator is used to remember the result of each function call.

When a previously calculated input is encountered, the cached result is returned instead of recalculating it.

This reduces the time complexity from exponential to linear.

6.8 Function Call Comparison

The following table gives the expected number of function calls for the implementations used in the program.

n	Naive Recursive Calls	Memoised Calls
10	177	11
20	21,891	21
25	242,785	26
30	2,692,537	31

The memoised version calculates each distinct input only once when the cache is initially empty.

The naive version repeats calculations for the same inputs.

6.9 RecursionError at n = 1000

Python limits the maximum recursion depth to prevent excessive stack usage.

The current recursion limit can be obtained using:

sys.getrecursionlimit()

Even though memoisation reduces repeated calculations, the recursive function still makes nested function calls.

For n = 1000, the recursion depth may exceed the permitted limit, resulting in a RecursionError.

6.10 Deployment Decision

The iterative approach is selected for large staircases.

The iterative solution stores only the previous two values and calculates the next value using a loop.

This avoids recursive stack limitations.

Increasing the recursion limit is not the preferred solution because excessive recursion can consume stack resources and may crash the interpreter.

6.11 Complexity Analysis

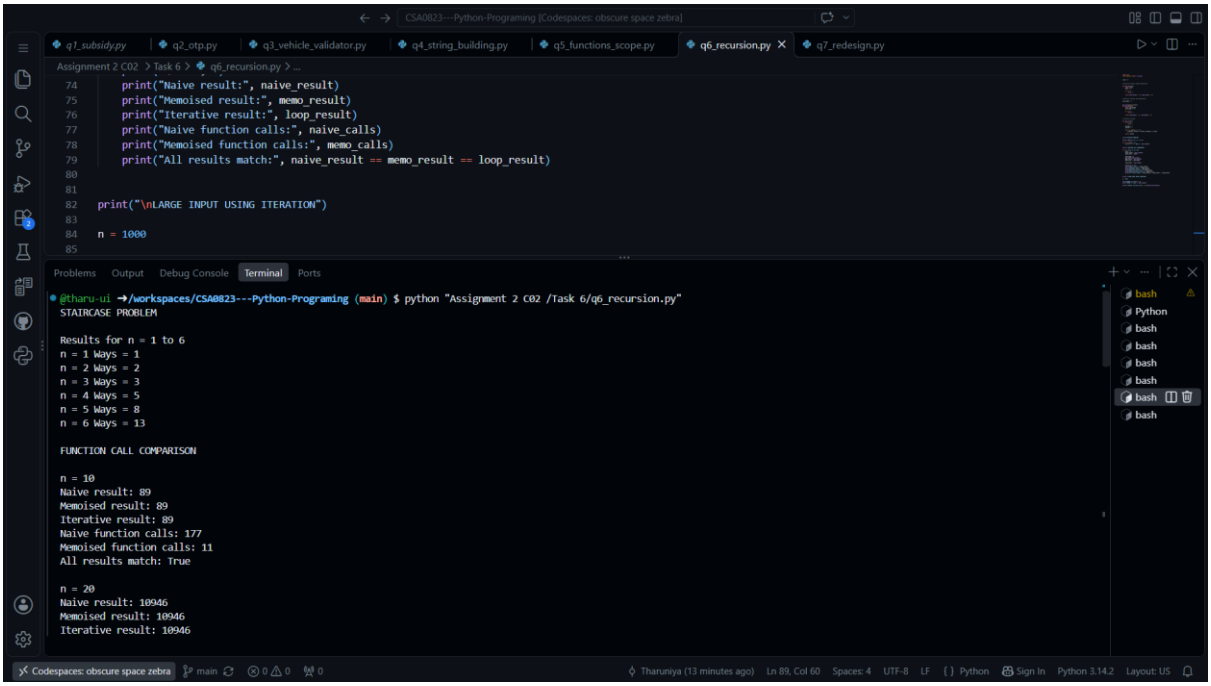
Method	**Time Complexity**	**Auxiliary Space**
Naive recursion	$O(2^n)$	O(n) stack
Memoised recursion	O(n)	O(n) cache and stack
Iterative approach	O(n)	O(1)

The iterative approach is most suitable for calculating staircase counts up to 1,000 steps.

6.12 Conclusion

The experiment demonstrates that naive recursion performs repeated calculations and becomes inefficient as the input grows. Memoisation reduces repeated work and improves time complexity to O(n). However, recursive calls still consume stack space. The iterative solution provides linear time complexity with constant auxiliary space and is therefore selected for deployment.

Output Screenshot :



QUESTION 7 – DIAGNOSE, REDESIGN AND VALIDATE

Marks: 13

7.1 Selected Program

The staircase counting program from Question 6 is selected for the design challenge.

The original naive recursive solution becomes inefficient when the number of staircase steps increases because it repeatedly calculates the same subproblems.

The redesigned solution uses memoisation to store and reuse previously calculated results.

7.2 A. Diagnose the Bottleneck

The main bottleneck in the original program is repeated recursive calculation.

For each input n, the function calls itself with n-1 and n-2.

The same smaller inputs are therefore calculated multiple times.

For example, when calculating ways(5), the program calculates ways(3) and ways(2) through multiple branches.

The number of function calls grows exponentially as the input increases.

The experimental evidence from Question 6 demonstrates this limitation.

Input	Naive Calls	Memoised Calls
10	177	11
20	21,891	21
25	242,785	26
30	2,692,537	31

This confirms that repeated calculations are the primary bottleneck.

7.3 B. Define the Design Objective

The objective is to reduce unnecessary recursive calculations while preserving the correctness of the staircase counting result.

The redesigned program should:

1. Produce the same result as the original program.
2. Reduce the number of function calls.
3. Improve execution time for larger inputs.
4. Avoid recalculating previously solved subproblems.

The measurable objective is to reduce the number of function calls from exponential growth to linear growth.

7.4 C. Design the Revised Solution

Memoisation is selected as the redesign technique.

A dictionary or cache stores the result of each calculated input.

When the same input is encountered again, the stored result is reused.

Property Exploited

The staircase problem has overlapping subproblems.

The same values of ways(n) are required multiple times during recursive execution.

Memoisation takes advantage of this property by storing the results.

Pseudocode

1. Define a function ways(n).
2. If n is a base case, return 1.
3. Check whether the result is already available in the cache.
4. If available, return the cached result.
5. Otherwise, calculate ways(n-1) + ways(n-2).
6. Store the calculated result in the cache.
7. Return the result.

Trade-off

Memoisation requires additional memory to store previously calculated results.

However, this extra memory significantly reduces repeated calculations.

The time complexity improves from $O(2^n)$ to $O(n)$.

7.5 D. Prediction Before Implementation

Before implementing the redesign, the following improvements are predicted:

- The number of function calls will decrease significantly.
- The execution time will decrease for larger inputs.
- Both programs will produce identical results.
- The redesigned solution will require additional memory for storing cached values.

The improvement will be measured using function-call counts and execution time.

7.6 E. Implementation and Validation

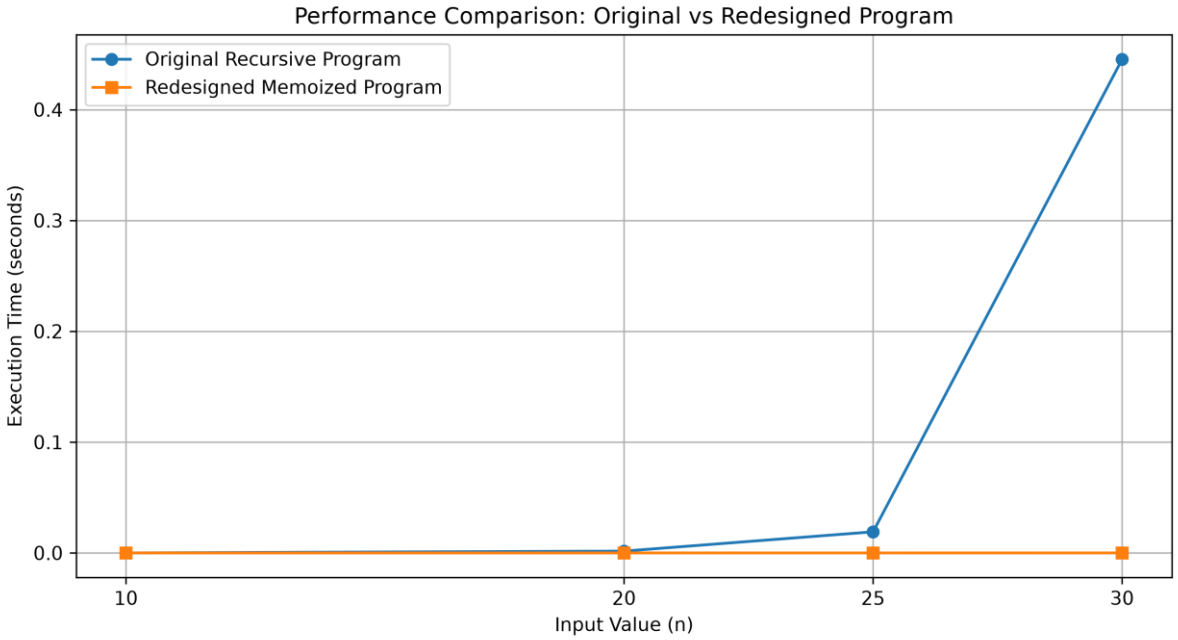
Both the original and redesigned programs are executed using the same input values.

The results are compared using:

1. Output correctness.
2. Number of function calls.
3. Execution time.

Experimental Results

n	Original Result	Original time (s)	Redesigned Result	Redesigned time (s)	Original Calls	Redesigned Calls
10	89	0.000021	89	0.000006	177	11
20	10,946	0.001633	10,946	0.000009	21,891	21
25	121,393	0.019056	121,393	0.000014	242,785	26
30	1,346,269	0.445328	1,346,269	0.000026	2,692,537	31



The graph compares the execution time of the original recursive program and the redesigned memoized program for different input values. The original recursive method takes significantly more time as the input increases because it repeatedly calculates the same values. The redesigned method uses memoization to store previously calculated results, reducing repeated calculations and execution time. Both methods produce identical results, but the redesigned method performs more efficiently.

Function Call Comparison

Input (n)	Original calls	Redesigned calls
10	177	11
20	21,891	21
25	242,785	26
30	2,697,537	31

Performance Improvement

Input (n)	Speed-up
10	3.50×
20	181.44×

25	1,361.14×
30	17,128×

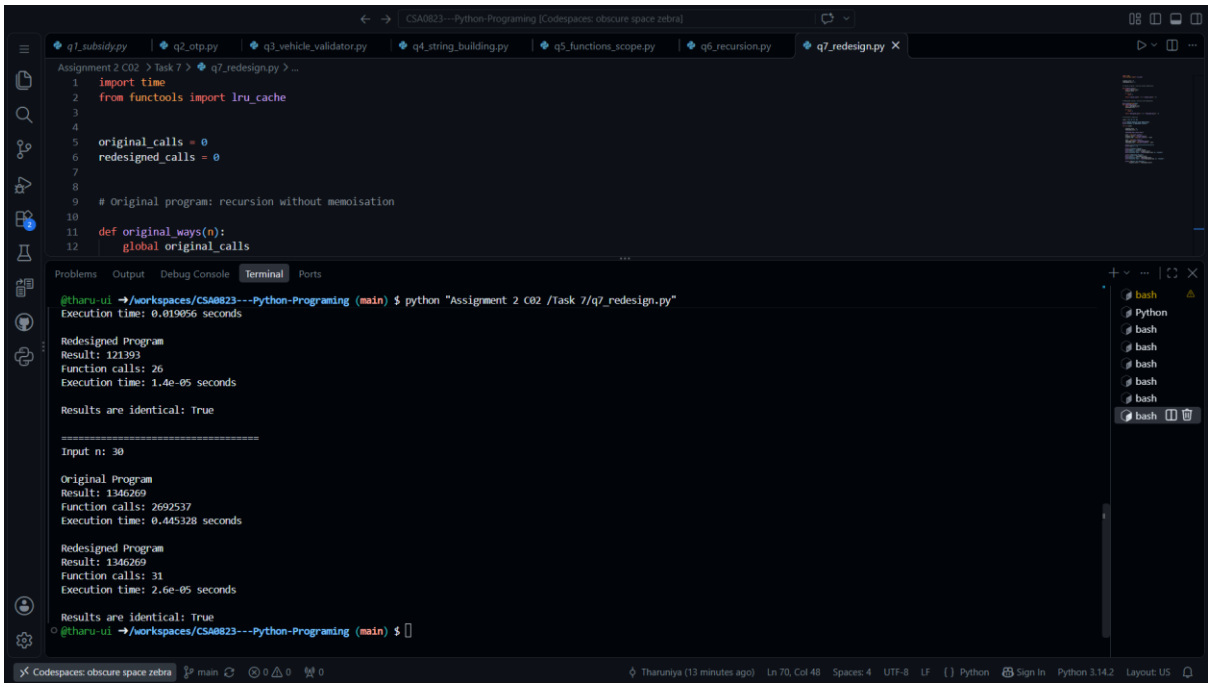
7.7 F. Defend the Final Design

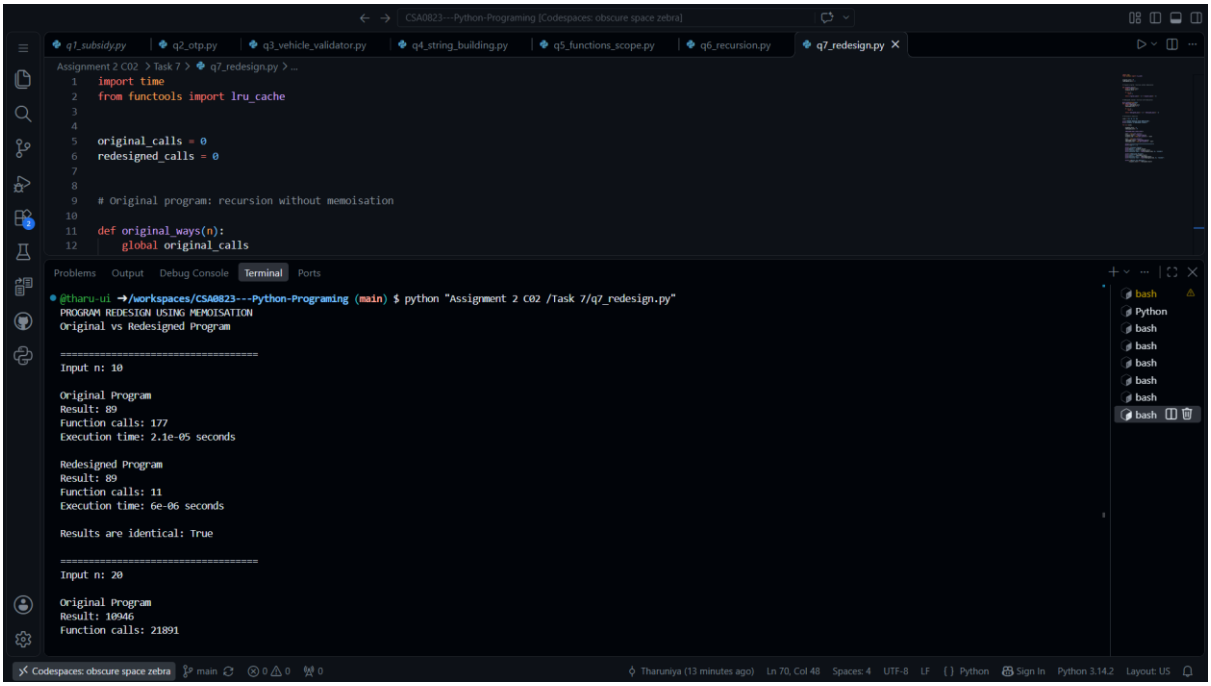
The redesigned solution successfully reduces repeated calculations by storing previously computed results.The original naive recursive solution is simple and useful for understanding recursion, but it is inefficient for larger inputs.The memoised solution is more appropriate when the same subproblems occur repeatedly.However, memoisation requires additional memory to store cached values.For very large inputs, an iterative approach may be even more suitable because it avoids recursion depth limitations and requires only constant auxiliary space.Therefore, memoisation is selected as the redesign for eliminating repeated work, while iteration is recommended for the final deployment of large staircase calculations.

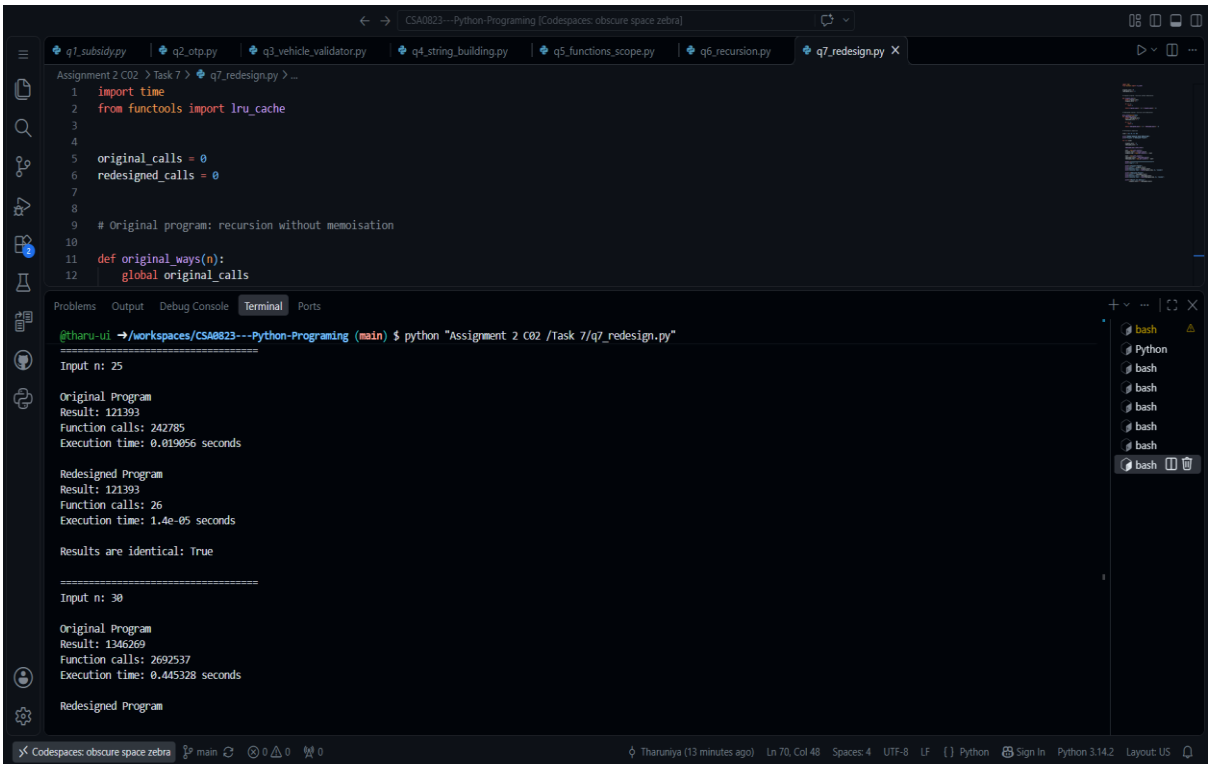
7.8 Final Conclusion

The design challenge demonstrates the importance of analysing a program before modifying it. The original staircase counting program performs unnecessary repeated calculations. Memoisation addresses this limitation by storing and reusing previously calculated results.The redesigned program preserves correctness while reducing the number of function calls from exponential growth to linear growth. Experimental validation through execution time and function-call measurements provides evidence of the improvement.

Output Screenshot




CSA0823---Python-Programing [Codespaces: obscure space zebra]
q1_subsidy.py
q2_otp.py
q3_vehicle_validator.py
q4_string_building.py
q5_functions_scope.py
q6_recursion.py
q7_redesign.py
Assignment 2 C02 > Task 7 > q7_redesign.py > ...
1 import time
2 from functools import lru_cache
5 original_calls = 0
6 redesigned_calls = 0
9 # Original program: recursion without memoisation
11 def original_ways(n):
12 global original_calls
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "Assignment 2 C02 /Task 7/q7_redesign.py"
PROGRAM REDESIGN USING MEMOISATION
Original vs Redesigned Program
==========
Input n: 10
Original Program
Result: 89
Function calls: 177
Execution time: 2.1e-05 seconds
Redesigned Program
Result: 89
Function calls: 11
Execution time: 6e-06 seconds
Results are identical: True
==========
Input n: 20
Original Program
Result: 10946
Function calls: 21891
bash
Python
bash
bash
bash
bash
bash
bash
Codespaces: obscure space zebra
main
Tharuniya (13 minutes ago)
Ln 70, Col 48
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US


CSA0823---Python-Programing [Codespaces: obscure space zebra]
q1_subsidy.py
q2_otp.py
q3_vehicle_validator.py
q4_string_building.py
q5_functions_scope.py
q6_recursion.py
q7_redesign.py
Assignment 2 C02 > Task 7 > q7_redesign.py > ...
1 import time
2 from functools import lru_cache
5 original_calls = 0
6 redesigned_calls = 0
9 # Original program: recursion without memoisation
11 def original_ways(n):
12 global original_calls
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "Assignment 2 C02 /Task 7/q7_redesign.py"
==========
Input n: 25
Original Program
Result: 121393
Function calls: 242785
Execution time: 0.019056 seconds
Redesigned Program
Result: 121393
Function calls: 26
Execution time: 1.4e-05 seconds
Results are identical: True
==========
Input n: 30
Original Program
Result: 1346269
Function calls: 2692537
Execution time: 0.445328 seconds
Redesigned Program
bash
Python
bash
bash
bash
bash
bash
bash
Codespaces: obscure space zebra
main
Tharuniya (13 minutes ago)
Ln 70, Col 48
Spaces: 4
UTF-8
LF
Python
Sign In
Python 3.14.2
Layout: US